Sports Academy Management System - Final Project Blueprint

Project Goal

Build a mobile application for a Cricket & Football Academy that manages:

- Student registrations
- Attendance
- Fee collection (Income)
- Expense management
- Monthly Profit & Loss
- Admin and Coach roles
- Future AI-powered automation

Current scale:

- Less than 100 students
- Single academy
- Android + iOS support
- Minimal operational cost

Final Tech Stack

Mobile App

Flutter

Why:

- Single codebase
- Android support
- iOS support
- Large ecosystem
- Easy Supabase integration

Backend

Supabase

Using:

- PostgreSQL Database
- Authentication

- Storage
- Realtime
- Edge Functions (later)

No separate backend required initially.

Avoid:

- Spring Boot
- NestJS
- Kubernetes
- Microservices

for Version 1.

Database

PostgreSQL (inside Supabase)

Stores:

- Students
- Attendance
- Payments
- Expenses
- Users

Authentication

Supabase Auth

Login via:

- Email + Password
  or
- Mobile OTP (future)

Storage

Supabase Storage

Buckets:

```
student-photos
expense-receipts
academy-documents
```

State Management

Riverpod

Recommended for Flutter.

Navigation

Go Router

Role-based navigation.

Reporting

PostgreSQL Views

Used for:

- Monthly Income
- Monthly Expenses
- Monthly P&L

AI Layer (Future)

OpenAI

Used for:

- WhatsApp Import
- Receipt Scanning
- Financial Insights
- Attendance Analytics

User Roles

Admin

Can:

- Add/Edit Students
- Mark Fees
- Add Expenses
- View Reports
- View P&L
- Manage Coaches

Coach

Can:

- View Students
- Mark Attendance
- View Attendance Reports

Cannot:

- View Expenses
- View P&L

Core Modules

Module 1 — Authentication

Screens:

```
Splash Screen
Login Screen
Forgot Password
```

Tables:

```
users
```

Module 2 — Student Management

Features:

- Add Student

- Edit Student
- Search Student
- Filter by Sport
- Student Profile

Student Fields:

```
Student Name
Parent Name
Phone Number
Age
Sport
Batch
Joining Date
Monthly Fee
Photo
Status
```

Module 3 — Attendance Management

Daily workflow:

```
Coach Login
↓
Select Batch
↓
Mark Attendance
↓
Submit
```

Features:

- Present
- Absent
- Monthly Attendance %
- Attendance Reports

Module 4 — Fee Management

Income tracking.

Features:

- Add Payment

- Payment History
- Monthly Fee Status
- Pending Fee Tracking

Fields:

```
Amount
Payment Date
Month
Year
Payment Mode
```

Module 5 — Expense Management

Expense Categories:

```
Cricket Pitch Construction
Football Pitch Construction
Shed Construction
Equipment
Ground Maintenance
Salary
Utilities
Miscellaneous
```

Features:

- Add Expense
- Upload Receipt
- View Expense History
- Category Filtering

Module 6 — P&L Dashboard

Admin Only.

Metrics:

```
Total Income
Total Expenses
Net Profit
```

Formula:

```
Profit = Income - Expenses
```

Do NOT store profit in database.

Calculate dynamically.

Flutter Screen Architecture

Splash

```
Splash
  ↓
Login
```

Admin Flow

```
Admin Dashboard
 │
 ├── Students
 │      ├── Student List
 │      ├── Student Details
 │      └── Add Student
 │
 ├── Attendance
 │      ├── Batch Selection
 │      ├── Mark Attendance
 │      └── Attendance Report
 │
 ├── Fees
 │      ├── Fee Overview
 │      ├── Student Payments
 │      └── Add Payment
 │
 ├── Expenses
 │      ├── Expense List
 │      ├── Add Expense
 │      └── Expense Detail
 │
 ├── Reports
 │      ├── Monthly P&L
 │      ├── Income Report
 │      └── Expense Report
 │
 └── Settings
```

Coach Flow

```
Coach Dashboard
|
├── Attendance
|
├── Student List
|
└── Profile
```

Database Design

users

```
id
name
phone
role
created_at
```

students

```
id
full_name
parent_name
phone
age
sport
batch
monthly_fee
join_date
status
photo_url
created_at
```

attendance

```
id
student_id
```

```
date
status
marked_by
created_at
```

Constraint:

```
One attendance record per student per day
```

---

payments

```
id
student_id
amount
payment_date
month
year
payment_mode
received_by
created_at
```

---

expense_categories

```
id
name
type
```

---

expenses

```
id
category_id
amount
expense_date
description
receipt_url
created_by
created_at
```

Supabase Storage Design

Bucket 1

```
student-photos
```

Bucket 2

```
expense-receipts
```

Bucket 3

```
documents
```

P&L Reporting Architecture

Create SQL Views.

View 1

```
monthly_income
```

View 2

```
monthly_expenses
```

View 3

```
monthly_pnl
```

Used directly by dashboard.

AI Roadmap (Future Versions)

AI Feature 1

WhatsApp Migration

Input:

```
Aryan Somkuwar
2500
05.06.2026
Cricket
Joined 30.05.2026
```

Output:

```json
{
  "student":"Aryan Somkuwar",
  "amount":2500
}
```

AI Feature 2

Receipt OCR

Upload:

```
Construction Receipt
```

AI extracts:

```
Vendor
Amount
Date
Category
```

and creates expense entry.

AI Feature 3

Finance Assistant

Questions:

```
Show expenses above ₹10000
```

```
Why was profit lower this month?
```

AI Feature 4

Attendance Intelligence

Identify:

```
Frequently absent students
```

```
Potential dropouts
```

Project Phases

Phase 1 (2–3 Weeks)

Foundation

- Supabase Setup
- Database Design
- Authentication
- Flutter Project Setup

Deliverable:

```
Working Login System
```

Phase 2 (2 Weeks)

Student Management

- Student CRUD
- Student Photos

Deliverable:

```
Student Module Complete
```

Phase 3 (1–2 Weeks)

Attendance

- Coach Dashboard
- Daily Attendance

Deliverable:

```
Attendance Module Complete
```

Phase 4 (1–2 Weeks)

Finance

- Fees
- Expenses

Deliverable:

```
Income + Expense Tracking
```

Phase 5 (1 Week)

Reports

- Monthly Income
- Monthly Expense
- Monthly P&L

Deliverable:

```
Management Dashboard
```

Phase 6 (Future)

AI

- WhatsApp Import

- Receipt OCR
- Financial Insights

Features Explicitly Deferred

Do NOT build initially:

- Parent App
- Tournament Management
- Performance Tracking
- Multi-Academy SaaS
- Microservices
- Kubernetes
- Complex AI Agents
- BigQuery/Data Warehouse

Final Architecture

```
Flutter App
        │
        ▼
Supabase Auth
        │
        ▼
PostgreSQL Database
        │
        ├── Students
        ├── Attendance
        ├── Payments
        ├── Expenses
        └── Reports
        │
        ▼
Supabase Storage
        │
        ▼
Future AI Layer
(OpenAI + Edge Functions)
```

This architecture is the most appropriate balance of **cost, simplicity, scalability, and maintainability** for an academy with fewer than 100 students while leaving room for future AI enhancements.